

Unit 1:
BIG DATA PROGRAMMING
TOOLS

Hbase Concepts

Overview

- ❑ Since 1970, RDBMS has been the solution for data storage and maintenance related problems.
- ❑ After the advent of big data, companies realized the benefit of processing big data and started opting for solutions like Hadoop.
- ❑ Hadoop uses distributed file system for storing big data, and MapReduce to process it.
- ❑ Hadoop excels in storing and processing huge data of various formats such as arbitrary, semi, or even unstructured.

Limitations of Hadoop

- ❑ Hadoop's core framework, MapReduce, is designed for batch processing large datasets. It's not suitable for real-time data analysis or streaming data. That means one has to search the entire dataset even for the simplest of jobs.
- ❑ A huge dataset when processed results in another huge data set, which should also be processed sequentially.
- ❑ At this point, a new solution is needed to access any point of data in a single unit of time (random access).

Hadoop Random Access Databases

- ❑ Applications such as HBase, Cassandra, couchDB, Dynamo, and MongoDB are some of the databases that store huge amounts of data and access the data in a random manner.

What is HBase?

- ❑ HBase is a distributed column-oriented database built on top of the Hadoop file system.
- ❑ It is an open-source project and is horizontally scalable.
- ❑ HBase is a data model that is similar to Google's big table designed to provide quick random access to huge amounts of structured data. It leverages the fault tolerance provided by the Hadoop File System (HDFS).
- ❑ It is a part of the Hadoop ecosystem that provides random real-time read/write access to data in the Hadoop File System.
- ❑ One can store the data in HDFS either directly or through HBase. Data consumer reads/accesses the data in HDFS randomly using HBase. HBase sits on top of the Hadoop File System and provides read and write access.

Why Choose HBase?

- ❑ A table for a popular web application may consist of billions of rows. If we want to search a particular row from such a huge amount of data, HBase is the ideal choice as query fetch time is less.
- ❑ Traditional relational data models fail to meet the performance requirements of very big databases. These performance and processing limitations can be overcome by Apache HBase.

Apache HBase Features

- ❑ HBase is linearly scalable.
- ❑ It has automatic failure support.
- ❑ It provides consistent read and writes.
- ❑ It integrates with Hadoop, both as a source and a destination.
- ❑ It has easy java API for client.
- ❑ It provides data replication across clusters.

Where to Use HBase

- ❑ HBase is suitable for scenarios requiring real-time, random access to large datasets.
- ❑ Common uses include time-series data storage (like IoT or financial data), real-time analytics (fraud detection, social media analysis), data caching, and metadata storage
- ❑ Companies such as Facebook, Twitter, Yahoo, and Adobe use HBase internally.
- ❑ In the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area.
- ❑ In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights.
- ❑ In sports, HBase is used to store match details and the history of each match. This data is then used for better predictions.

Key Concepts in HBase

- **Column-oriented Storage:** Unlike traditional relational databases, HBase stores data in a column-oriented fashion, optimizing for fast retrieval of specific columns or column families.
- **Row Key:** Each row in an HBase table has a unique row key, which serves as the primary identifier and determines the sorting order of data.
- **Column Families:** These are logical groupings of related columns. They are defined during table creation and cannot be changed easily.
- **Column Qualifiers:** These are specific identifiers within a column family, providing granular access to data. Column qualifiers are mutable and can vary between rows.
- **Cell:** A cell represents the intersection of a row key, column family, and column qualifier, containing a value and a timestamp (version).
- **Timestamp/Versioning:** HBase automatically assigns a timestamp to each cell value, enabling the storage and retrieval of multiple versions of the same data.

Storage Mechanism in HBase

- HBase is a column-oriented database value and the tables in it are sorted by row.
- The table schema defines only column families, which are the key value pairs.
- A table can have multiple column families and each column family can have any number of columns.
- Subsequent column values are stored contiguously on the disk.
- Each cell value of the table has a timestamp.

In HBase:

- **Table is a collection of rows.**
- **Row is a collection of column families.**
- **Column family is a collection of columns.**
- **Cell / Key-Value : Stores the actual data value along with a timestamp.**

[illegible]

Examples:



Each cell has multiple versions, typically represented by the timestamp of when they were inserted into the table

Timestamp1 Timestamp2

Row Key	Column Family - Personal		Column Family - Office	
	Name	Residence Phone	Phone	Address
00001	John	415-111-1234	415-212-5544	1021 Market St
00002	Paul	408-432-9922	415-212-5544	1021 Market St
00003	Ron	415-993-2124	415-212-5544	1021 Market St
00004	Rob	815-243-9988	408-998-4322	4455 Bird Ave
00005	Carly	206-221-9123	408-998-4325	4455 Bird Ave
00006	Scott	818-231-2566	650-443-2211	543 Dale Ave

The table is lexicographically sorted on the row keys

Cells

COLUMN FAMILIES

Row key	personal data		professional data	
empid	name	city	designation	salary
1	raju	hyderabad	manager	50,000
2	ravi	chennai	sr.engineer	30,000
3	rajesh	delhi	jr.engineer	25,000

Row Key

Column Family

Row Key	Customers		Products	
Customer ID	Customer Name	City & Country	Product Name	Price
1	Sam Smith	California, US	Mike	\$500
2	Arijit Singh	Goa, India	Speakers	\$1000
3	Ellie Goulding	London, UK	Headphones	\$800
4	Wiz Khalifa	North Dakota, US	Guitar	\$2500

Column Qualifiers

Cell

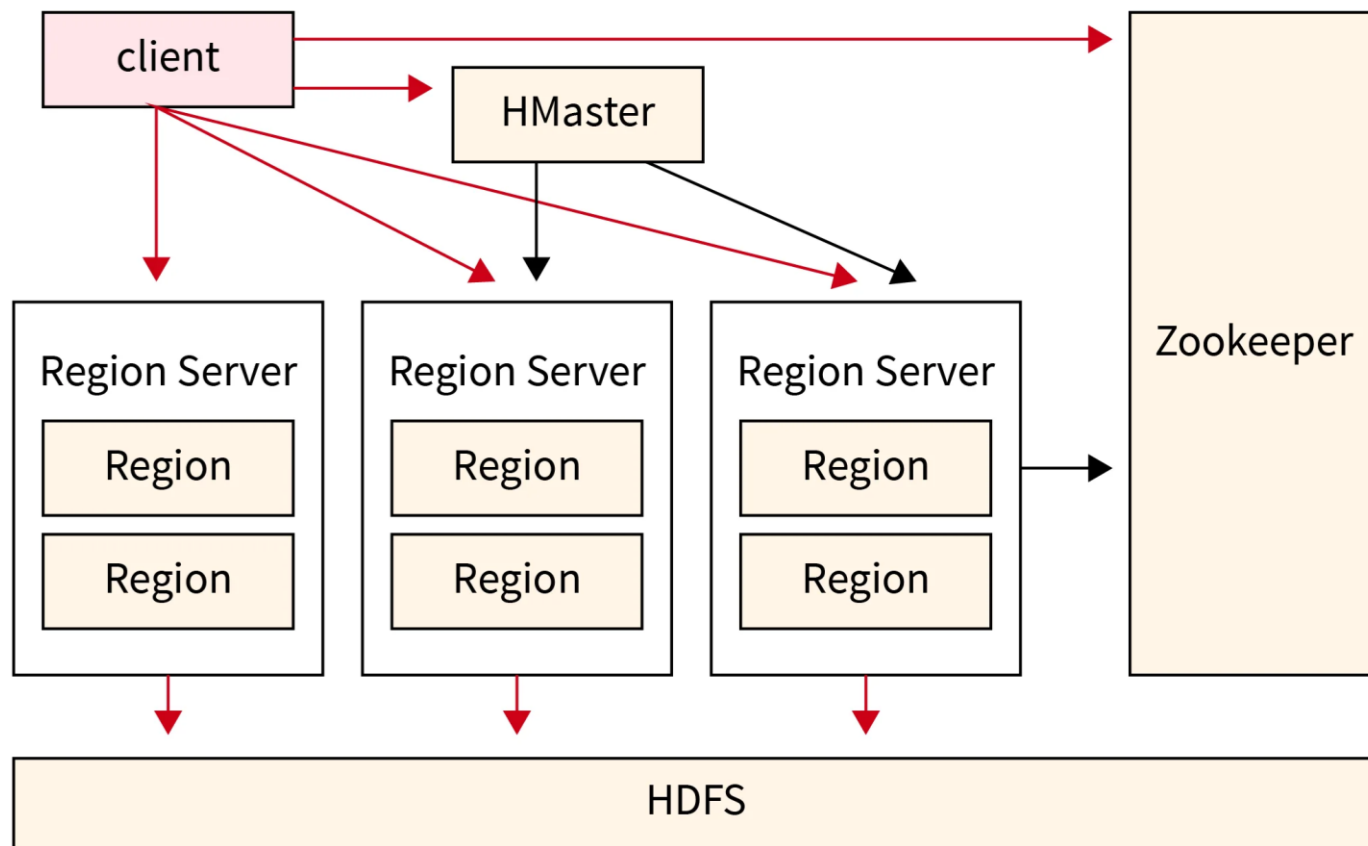
How is HBase different from other NoSQL models

- HBase stores data in the form of key/value pairs in a columnar model.
- All the columns are grouped together as Column families.
- HBase provides a flexible data model and low latency access to small amounts of data stored in large data sets.
- HBase on top of Hadoop increases the throughput and performance of distributed cluster set up.
- It provides faster random reads and writes operations.

HBase Vs. RDBMS

Feature / Aspect	Apache HBase	Traditional RDBMS
Data Architecture	Column-oriented datastore	Row-oriented datastore
Schema Requirement	Schema-less (only column families defined upfront)	Fixed / rigid schema required
Table Structure	Wide, sparse tables designed for de-normalized data	Thin tables designed for normalized data
Data Types Supported	Structured and semi-structured data	Structured data
Partitioning & Scaling	Built-in automatic partitioning (horizontally scalable)	No built-in support for automatic partitioning (vertically scalable)

HBase - Architecture



Apache HBase Architecture

HBase - Architecture

Key components

- **Hmaster**
- **RegionServers**
- **ZooKeeper**

HBase - Architecture

Key components

- **HMaster:**

- Single master Node running at a time
- Manages Cluster Operations
- The master server manages metadata, assigns regions to RegionServers, and handles administrative tasks like table creation, deletion, and schema modifications.
- It monitors RegionServer health and performs load balancing.

Note : Regions are logical divisions of an HBase table, containing a sorted range of rows.

HBase - Architecture

Key components

- **RegionServers:**

- These act as worker nodes, handling read/write requests from clients.
- They host HBase table regions, which are horizontal partitions of data based on row keys.
- RegionServers also manage data within those regions.
- Each RegionServer operates on a DataNode within the Hadoop cluster.

- **ZooKeeper:**

- This acts as a coordinator, maintaining cluster state, facilitating communication between HMaster and RegionServers, and handling server failure detection and recovery.
- It tracks active HMaster and RegionServers and provides server failure notifications.

Commands

1. General Commands

Command	Use / Description	Syntax	Example
help	Displays help information and usage syntax for HBase commands.	<code>help ['command_name']</code>	<code>help</code> <code>help 'create'</code>
status	Displays cluster operational status, including server count, dead servers, and average load.	<code>status</code> <code>['simple' 'summary' 'detailed']</code>	<code>status</code> <code>status 'simple'</code>
version	Displays the current running version of Apache HBase.	<code>version</code>	<code>version</code>
whoami	Displays information about the currently authenticated user and active roles.	<code>whoami</code>	<code>whoami</code>

2. Data Definition Language (DDL) Commands

Command	Use / Description	Syntax	Example
create	Creates a new table with one or more specified column families.	create 'table_name', 'col_family1' [, 'col_family2', ...]	create 'emp', 'personal_data', 'professional_data'
list	Lists all user-defined tables present in the HBase database.	list ['regex_pattern']	list list 'e.*'
describe	Provides detailed schema information and configuration properties of a table.	describe 'table_name'	describe 'emp'
alter	Modifies table schema, such as adding/removing column families or changing version history.	alter 'table_name', {NAME => 'col_family', KEY => 'value'}	alter 'emp', NAME => 'financial_data' alter 'emp', NAME => 'personal_data', VERSIONS => 3
disable	Disables a table to prevent data operations and prepare it for structural changes or dropping.	disable 'table_name'	disable 'emp'
enable	Enables a previously disabled table to resume read and write operations.	enable 'table_name'	enable 'emp'
is_disabled	Checks whether a specified table is currently disabled.	is_disabled 'table_name'	is_disabled 'emp'
is_enabled	Checks whether a specified table is currently enabled.	is_enabled 'table_name'	is_enabled 'emp'
drop	Deletes a table from HBase. The table must be disabled first.	drop 'table_name'	disable 'emp' drop 'emp'

3. Data Manipulation Language (DML) Commands

Command	Use / Description	Syntax	Example
put	Inserts or updates a cell value at a specified row key, column family, and column qualifier.	put 'table_name', 'row_key', 'col_family:qualifier', 'value'	put 'emp', 'row1', 'personal_data:name', 'John Doe' put 'emp', 'row1', 'personal_data:city', 'Ahmedabad'
get	Retrieves row contents or specific cell values for a given row key.	get 'table_name', 'row_key' [, {COLUMNS => ['cf:q']}]	get 'emp', 'row1' get 'emp', 'row1', {COLUMN => 'personal_data:name'}
scan	Scans and returns table data with optional row ranges, limits, or column filters.	scan 'table_name' [, {LIMIT => n, STARTROW => 'r1', STOPROW => 'r2'}]	scan 'emp' scan 'emp', {COLUMNS => ['personal_data:name'], LIMIT => 5}
delete	Deletes a specific cell (column value) within a row key.	delete 'table_name', 'row_key', 'col_family:qualifier'	delete 'emp', 'row1', 'personal_data:city'
deleteall	Deletes all cells in all column families for a given row key.	deleteall 'table_name', 'row_key'	deleteall 'emp', 'row1'
count	Counts the total number of rows in a table.	count 'table_name' [, INTERVAL => n, CACHE => m]	count 'emp' count 'emp', INTERVAL => 1000
truncate	Clears all data from a table by disabling, dropping, and recreating the original schema.	truncate 'table_name'	truncate 'emp'
incr	Increments a counter value in a specific cell by a specified amount.	incr 'table_name', 'row_key', 'col_family:qualifier', amount	incr 'emp', 'row1', 'stats:login_count', 1

4. Tools and Admin Commands

- `major_compact 'table_name'`: Initiates a major compaction for a table.
- `flush 'table_name'`: Forces a flush of in-memory data to HFile.
- `snapshot 'table_name', 'snapshot_name'`: Creates a snapshot of a table.
- `restore_snapshot 'snapshot_name'`: Restores a table from a snapshot.